Text-to-SQL con LoRA

Proyecto 3 · Deep Learning 2026 · Universidad del Valle

Pablo · Luis Palacios

Qwen2.5-Coder-1.5B-Instruct + adapter LoRA (fp16)

Esquema (CREATE TABLE con COMMENT) + pregunta en español -> MySQL 8.4

Problema y contrato

generar_sql(esquema, pregunta, ejecutar=None) -> una sola SELECT / WITH

Generación determinista (do_sample=False, semilla 2026)

config.json: usar_adapter true/false con el MISMO prompt

Salida filtrada a solo lectura (nunca modifica la base)

Métrica: EX ponderada (fácil 1, media 2, difícil 3)

Dataset de entrenamiento

2,099 pares pregunta–SQL sobre 10 esquemas propios

fácil: 1024 · media: 539 · difícil: 536

Nombres en español e inglés, con abreviaturas (hr, fleet, clinic...)

Plantillas por nivel: filtros, JOIN, GROUP BY/HAVING, subconsultas,

WITH, funciones de ventana y fechas

Pares validados ejecutándose en MySQL 8.4.10 (ONLY_FULL_GROUP_BY)

Validación propia

2 esquemas NO vistos en entrenamiento: aerolinea y banco

40 preguntas escritas por el equipo, en los tres niveles

Partición por esquema, no por pregunta (evita fuga de esquema)

Harness (src/evaluar.py) replica las reglas del evaluador:

- orden solo si se pide, redondeo a 2 decimales, columna extra no penaliza

Configuraciones LoRA (un solo factor: rank r)

Mismo lr (2e-4), dropout 0.05, alpha/r = 2, 100 pasos, batch 2x8 acum., q,k,v,o,gate,up,down

Loss solo sobre la respuesta SQL (prompt enmascarado)

Hipótesis A (r=8): rank bajo basta para el patrón SQL y entrena más rápido

Hipótesis B (r=32): más capacidad ayuda en consultas difíciles

(subconsultas, ventanas), a costa de más parámetros y tiempo

Resultados de entrenamiento

Config A: r=8, alpha=16

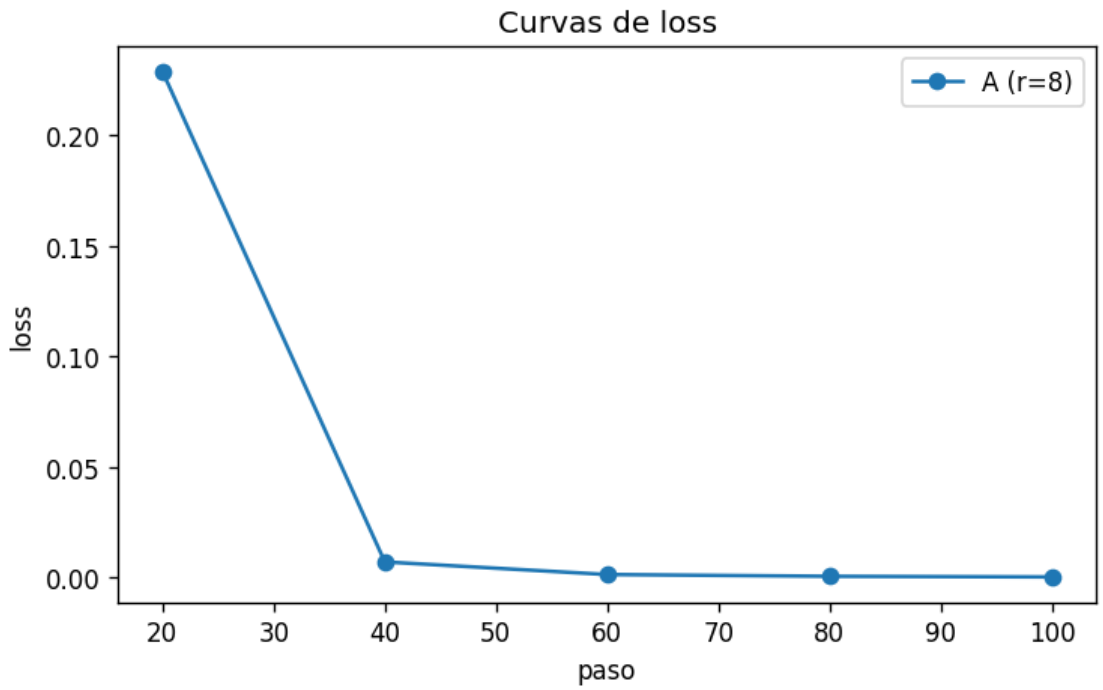
- parámetros entrenables: 9,232,384 (0.5945% del total)
- tiempo de entrenamiento: 9.0 min (T4, LoRA fp16)
- pasos: 100, pares: 2099, loss final: 0.000

Config B: no se completó (la GPU T4 seguía ocupada tras A; sin tiempo para reintentar)

- la hipótesis de B queda sin contrastar; se entrega A

Curvas de loss

Loss de entrenamiento por paso



Baseline vs adapter (validación propia)

EX en validación propia: ver resultados/validacion_propia.csv

(notebook 03_evaluacion.ipynb con MySQL 8.4.10)

Experimento de inferencia: auto-corrección

Si el evaluador entrega ejecutar(sql), se ejecuta la consulta generada

Si MySQL devuelve error, se re-genera con el mensaje de error como pista

Máximo 3 llamadas a ejecutar por pregunta; siempre se devuelve SELECT

Ataca errores de sintaxis y columnas/tablas inexistentes,

- no corrige errores lógicos (resultado distinto)

Análisis de errores por categoría

Categorías: sintaxis, columna/tabla inexistente, JOIN, agregación,

filtro, resultado distinto

Esperado: el baseline falla más por formato y nombres inventados;

el adapter aprende el formato y a usar los COMMENT del esquema

Lo difícil (ventanas, fechas, WITH) concentra los errores restantes

Predicción justificada de EX (evaluación oculta)

Predicción con adapter: EX ~ 0.45–0.55; sin adapter: ~ 0.30–0.40

Justificación:

- el modelo base 1.5B ya resuelve bien preguntas de una tabla (peso 1)
- el adapter mejora formato, uso de COMMENT y JOIN/GROUP BY (peso 2)
- las difíciles (peso 3) siguen siendo el cuello de botella
- esquemas ocultos con nombres abreviados -> algo de caída vs validación
- entrenamiento corto (100 pasos, ~0.8 épocas) limita la mejora
- la loss baja a ~0.01: los pares de plantilla se memorizan rápido,
  riesgo de sobreajuste al estilo de nuestros esquemas

Conclusiones y trabajo futuro

Pipeline completo: datos validados, LoRA A/B, harness EX, auto-corrección

Siguiente paso: más épocas, más datos difíciles (Spider/BIRD traducidos),

few-shot con ejemplos del mismo esquema

Base para el Proyecto Final: adapter, prompt, dataset y evaluador